화면에서 배포까지, 경험으로 연결한 개발

바흐티요르 파르피보예프 | Bakhtiyor Parpiboev

React, TypeScript와 Node.js, Express를 중심으로 사용자 화면, 서버 API, 데이터베이스를 연결하는 소프트웨어 엔지니어입니다. 프리랜서 협업과 개인, 팀 프로젝트에서 AI 기능을 사용자 작업 흐름에 연결하고, 인증, 데이터 저장, 예외 처리, 배포를 경험했습니다.

기능이 실행되는 것에 더해 사용자가 결과를 수정하고, 실패 상황을 이해하며, 다음 작업을 이어갈 수 있도록 구현해 왔습니다. 전공 지식과 도메인 지식을 바탕으로 요구사항 이해부터 설계, 구현, 검증, 개선까지 연결하는 사용자 중심의 앱과 웹을 개발하고자 합니다.

이 문서에서 확인할 수 있는 경험

- 실무 협업: UITC의 Express 서버, 프론트엔드 연동 및 학습 지원 챗봇 공동 개발
- 개인 개발: AI Web Builder, bakhtiyor.me, MDB Academy, AI Pass
- 팀 개발: Friendly의 서버, 인증, 파인튜닝, 챗봇, Dojang의 공지, 시간표, 학업, PWA

학력 및 배경

우즈베키스탄 출신으로 대한민국 정부초청장학금(GKS) 장학생에 선발되어 국민대학교 소프트웨어학부에 재학 중입니다.

Website / bakhtiyor.me

GitHub / BakhtiyorParpiboev

작성 기준: 2026.11.05. 개인 담당 업무와 팀 전체 기능을 구분했습니다. 제공된 소스, 사용자 확인, 기존 검토 기록을 바탕으로 작성했으며, 데모 링크의 모든 기능이 현재 실행됨을 보증하는 문서는 아닙니다.

UITC

4인 프리랜서 팀 | 참여 기간 3개월

서버 개발에서 AI 기능 연동으로

UITC에서는 4인 프리랜서 팀의 일원으로 3개월간 참여했습니다. 초기 2개월 동안 Express.js 기반 서버 개발과 프론트엔드 연동을 진행했으며, 마지막 한 달에는 팀원들과 OpenAI 모델 기반 AI 학습 지원 챗봇을 개발했습니다.

백엔드 또는 프론트엔드만 다루던 작은 프로젝트 경험에서 나아가, 화면의 사용자 입력이 서버 요청으로 전달되고 처리 결과가 다시 화면에 반영되는 통합 개발을 경험했습니다. 팀이 작성한 코드와 연결되는 지점을 확인하며 기능을 완성하는 과정에 참여했습니다.

직접 수행한 업무

- Express.js 기반 서버 개발 및 프론트엔드와의 API 연동
- 팀원들과 OpenAI 모델 기반 학습 지원 챗봇 공동 개발
- 프론트엔드, 서버의 요청 및 응답 흐름을 연결하는 협업

기술 스택

Node.js , Express.js , 프론트엔드 API 연동 , OpenAI API

경험의 의미

실무 협업을 통해 화면과 서버를 별개의 구현 대상으로 보기보다 하나의 사용자 흐름으로 연결하는 관점을 쌓았습니다. 이후 개인, 팀 프로젝트에서도 인증, API, 데이터 저장과 화면 상태를 함께 고려하는 기반이 되었습니다.

참여 연월과 외부 공개 결과물은 이 문서에 기재하지 않았습니다. 다른 프로젝트와의 관계가 확인되지 않아 별도 프로젝트 수나 누적 경력 연수로 환산하지 않았습니다.

AI Web Builder

AI 생성과 직접 편집을 결합한 웹 제작 도구 | 개인 프로젝트 , 베타

자연어로 원하는 웹사이트를 설명하면 AI가 초안을 생성하고, 사용자가 미리보기에서 결과물을 직접 다듬는 웹 서비스입니다. 작은 수정마다 프롬프트를 다시 작성하고 기다리는 부담을 줄이고, 사용자가 만드는 과정에서 선택권과 주도감을 느끼도록 설계했습니다.

생성부터 저장까지 연결한 구현

- 인증과 잔액 확인 후 5크레딧을 차감하고, 프로젝트, 대화 기록을 저장한 뒤 생성 작업을 진행했습니다.
- OpenAI SDK를 OpenRouter에 연결해 NVIDIA Nemotron 계열 모델을 호출하고, 요구사항 보강과 HTML 생성, 수정 흐름을 구성했습니다.
- 생성 코드, 대화, 버전을 PostgreSQL에 저장하고 폴링으로 진행 상태를 표시했습니다.
- 텍스트, 색상, 간격, 글자 크기 직접 편집, 반응형 미리보기, 버전 저장, 복원, 프로젝트 공개, 비공개 기능을 개발했습니다.

인증과 테스트 결제

Better Auth로 인증을 연결하고 Stripe Checkout 세션 생성 및 웹훅을 통한 크레딧 충전 흐름을 구현했습니다. 현재 베타의 결제는 테스트 모드로, 실제 금전 거래와 구분합니다. 생성 결과는 HTML 기반 웹사이트이며 사용자 서비스의 백엔드, DB를 자동 생성하는 기능은 아닙니다.

기술 스택

React , TypeScript , Vite , Tailwind CSS , Axios , Node.js , Express , PostgreSQL , Prisma , Better Auth , OpenAI SDK , OpenRouter , Stripe

구현 코드 / AI Web Builder

AI Web Builder | 문제 해결

크레딧의 일관성과 직접 편집 결과의 저장

01. 실패한 요청의 크레딧 반환

서버 연결 중단, 웹사이트 생성이 15분 이상 지연되는 상황, 모델 제공자의 오류 등에서 사용자가 결과를 받지 못했는데 크레딧이 차감되는 문제가 있었습니다. 사용자 입장에서 여러 번 실행하며 실패 가능 지점을 찾고, 해당 실패 경로에서 차감된 5크레딧을 반환하는 처리를 추가했습니다.

이 과정에서 정상 흐름뿐 아니라 예상하지 못한 시나리오를 직접 재현하는 테스트의 중요성을 배웠습니다. 모든 장애 상황에서의 정합성을 보장한다고 주장하기보다, 확인한 실패 경로를 기준으로 반환 처리를 보완한 경험입니다.

02. 화면 요소 선택과 변경 결과 저장

직접 편집의 목표는 분명했지만 화면 요소를 선택하고, 수정 결과를 데이터베이스에 반영하는 구현 방법을 찾는 데 약 일주일을 사용했습니다. 요소 선택 방식은 시니어 프론트엔드 개발자에게 조언을 구해 마우스, 호버 상호작용을 이해하며 해결했습니다.

저장 로직은 기존 처리를 복잡하게 확장하기보다 수정 코드를 업데이트하는 별도 백엔드 함수로 분리했습니다. 화면 조작과 서버 저장의 책임을 나누며 변경 흐름을 정리했습니다.

학습과 다음 개선

- 사용자 관점의 예외 시나리오로 실패 지점을 찾고, 처리 책임을 작은 함수로 분리
- 동작하는 프로토타입을 먼저 만들고 피드백으로 반복 개선
- AI 프롬프트 품질 개선, Stripe 실제 결제 검증 및 OpenRouter 모델별 동작 비교

Friendly

학습 흐름을 연결한 5인 팀 프로젝트 | 개인 기여 중심

Friendly는 시간표와 일정, 학습 자료, 강의 녹음 및 학습 지원 챗봇을 연결하는 서비스입니다. 팀 전체 기능 중 저는 서버 구성과 인증, 파인튜닝 데이터 준비, 학습 실행, 모델 연동, 챗봇 API 및 화면, 캘린더, 튜토리얼 UI를 담당했습니다.

직접 수행한 구현

- Express 서버에 인증, 사용자, 일정, 캘린더, 챗봇 등의 라우트를 연결하고 Firebase ID 토큰 검증과 사용자 프로필 흐름을 구현했습니다.
- 다국어 질의응답 약 2,000개를 작성, 정제하고 OpenAI 학습용 대화 형식으로 구성했습니다. 파인튜닝 작업 실행과 모델 연동을 담당했으며 결과 평가는 담당 범위에 포함하지 않습니다.
- 챗봇 API의 입력 검증, 타임아웃, 오류 응답과 화면의 메시지, 전송 중 상태를 연결했습니다.
- 캘린더, 목록 UI, 튜토리얼, 공통 안내 화면을 구현하고 Postman으로 API를 테스트했습니다.

팀 아키텍처를 이해하며 연결한 범위

클라이언트는 React Native, Expo, 서버는 Express를 사용하며 Firebase 인증과 Firestore를 연결합니다. 팀 통합 기능에는 시간표 이미지 분석, Google Calendar 연동, 강의 녹음의 전사, 요약도 포함됩니다. 이 기능 전체를 개인 구현으로 주장하지 않습니다.

기술 스택

React Native , Expo , TypeScript , Node.js , Express , Firebase Authentication , Firestore , OpenAI API , Postman

팀 저장소 / Friendly

Friendly | 모델과 협업

학습 데이터 준비, 응답 범위 조정, 브랜치 통합

01. 데이터 준비와 모델 응답 범위

질의응답 형태의 다국어 데이터를 수집, 작성하고 정제해 학습에 사용했습니다. 파인튜닝 과정에서는 모델의 말투와 역할이 원하는 방향으로 유지되지 않거나 프로젝트와 관계없는 주제에 답변하는 문제가 있었습니다.

모델에 시스템 지시문을 제공하고 서비스 범위 밖 질문에 대한 답변을 제한하는 방향으로 조정했습니다. 데이터만 준비하는 것으로 끝나는 것이 아니라, 호출 시의 지시문과 실제 서비스에서 기대하는 역할을 함께 설계해야 한다는 점을 배웠습니다. 정량 평가를 수행한 경험은 아니므로 정확도나 개선율은 기재하지 않습니다.

02. 5인 개발에서의 변경 사항 통합

여러 팀원이 동시에 코드를 작성하며 브랜치와 변경 사항을 합치는 과정에서 충돌을 경험했습니다. 작업용 브랜치를 별도로 만들고 해당 브랜치에 변경 내용을 올린 뒤, 팀 리더가 주요 브랜치로 통합하는 방식으로 협업했습니다.

이를 통해 개인 코드의 완성뿐 아니라 다른 사람이 검토하고 합칠 수 있는 단위로 작업을 정리하는 중요성을 배웠습니다. 팀 리더의 최종 통합 역할과 제가 수행한 작업 브랜치 개발을 구분합니다.

검증과 저장 경로에 대한 이해

Postman으로 요청과 응답을 확인하며 화면 문제와 API 문제를 구분했습니다. 저장 위치는 기능별로 다릅니다. 팀 통합 코드와 웹 QA 경로에 차이가 있어, 기기 내 저장, Firestore, 서버 파일을 모두 하나의 저장소처럼 설명하지 않도록 구분했습니다.

서버, 학습 데이터, 화면 구현 확인

Dojang

외국인 유학생의 행정, 학업 지원 | 5인 팀 프로젝트

한국에 거주하는 외국인 유학생이 필요한 학교 정보와 학업 기능을 한곳에서 확인하도록 만든 서비스입니다. 학생 대상 설문조사에 참여하고 학교 공지, 시간표, 수강, 학업, 리마인더와 PWA를 담당했습니다.

직접 수행한 업무

- 공개 학교 공지를 수집하는 API와 공지 목록 화면을 연결했습니다. 학교 포털 계정이나 비공개 학사 시스템을 연동한 것은 아닙니다.
- 시간표 API, 화면을 구현하고 일정 데이터를 사용자 화면과 연결했습니다.
- 학점, GPA, 학업 요건 계산 API와 수강, 학업 화면을 개발했습니다. 계산 함수와 데이터 흐름을 화면까지 연결했습니다.
- 리마인더, 체크리스트, manifest와 Service Worker 기반 PWA, 오프라인 대체 화면을 구현했습니다.

구현을 설명하는 기준

팀 서비스의 AI 검색, 행정 안내 전체를 개인 기여로 묶지 않고, 담당한 기능과 확인 가능한 커밋으로 역할을 제시합니다. 시간표, 학업 정보 일부는 데모 데이터이며 외부 메신저, 이메일, 푸시 발송 완료를 주장하지 않습니다. 실제 겪지 않은 장애나 문제 해결 사례도 추가하지 않았습니다.

기술 스택

React , TypeScript , Node.js , Express , REST API , 브라우저 저장소 , PWA , Service Worker

서비스 데모 / Dojang

팀 저장소

학업 계산, API, 화면 구현 커밋

bakhtiyor.me

콘텐츠 관리부터 배포, 검색 접근성까지 | 개인 웹사이트

프로젝트와 블로그, 프로필을 직접 관리하는 개발자 웹사이트입니다. React, TypeScript 화면과 Express API, PostgreSQL을 연결하고 Vercel 프론트엔드, Render API, Neon 데이터베이스를 활용한 배포를 경험했습니다.

콘텐츠와 접근 권한

- 프로젝트, 블로그, 프로필 관리와 공개, 초안 상태를 연결했습니다.
- Clerk 인증에 더해 서버에서 소유자의 검증된 이메일을 확인하는 편집 권한 검증을 구성했습니다.
- OpenAPI 계약과 생성된 클라이언트, 검증 구조를 활용해 화면과 API의 데이터 형식을 연결했습니다.

기술적 SEO와 검색 등록

페이지별 제목, 설명, canonical URL, Open Graph와 Person 구조화 데이터를 정비하고 sitemap, robots.txt를 구성했습니다. 주요 공개 경로에는 빌드 시 메타데이터 HTML을 생성하고, 관리자, 인증 경로는 검색 제외 정책을 적용했습니다.

Google Search Console에서는 DNS TXT 기반 도메인 인증, 사이트맵 제출과 홈페이지 색인 요청을 진행했습니다. 이는 검색 접근성과 등록 절차에 대한 경험이며, 실제 검색 순위나 방문자 증가 성과를 뜻하지 않습니다. 문서 기준 일부 개선의 배포 상태는 별도 확인이 필요합니다.

자동 검증과 남은 범위

GitHub Actions에서 타입 검사, 테스트, 빌드, SEO 산출물 검사를 실행하도록 구성했습니다. 본문 전체의 서버 렌더링은 구현 범위가 아니며, 게시글 본문은 JavaScript와 공개 API에 의존합니다.

운영 웹사이트

코드 / 배포, CI 설정

SEO 검토와 Search Console 기록

MDB Academy

우즈베크어 프로그래밍 학습 플랫폼 | 개인 프로젝트

AI 질의응답, 프로그래밍 게임, 학습 로드맵과 코드 실행 환경을 하나로 연결한 학습 서비스입니다. 학습자가 질문하고 코드를 실행하며 자신의 활동을 확인할 수 있도록 프론트엔드와 서버를 함께 개발했습니다.

학습 기능과 데이터 연결

- Firebase 인증과 Express 사용자 동기화, PostgreSQL, Prisma 기반 프로필, 로드맵 저장을 구현했습니다.
- Gemini 챗봇의 대화 생성, 목록, 조회, 삭제와 메시지, 대화 문맥 저장을 개발했습니다.
- 퀴즈, 버그 찾기, 코드 완성, 출력 예측, 코드 순서 배열, 카드 맞추기 등 6종의 게임을 구현했습니다.
- XP, 레벨, 연속 학습, 리더보드 API와 프로필 활동 집계를 연결했습니다.
- Monaco 편집기에서 언어, 파일, 표준입력을 선택하고 결과, 오류를 확인하는 화면과 서버 실행 경로를 구성했습니다.

구현 범위와 학습

JavaScript, Python, C, C++, Java, Go 실행 경로를 구현했으며 실제 실행에는 서버의 해당 런타임, 컴파일러가 필요합니다. 커뮤니티는 준비 중 화면이며, 완료된 강의 수나 실제 이용자 수를 성과로 제시하지 않습니다. 인증, 학습 활동, AI 대화, 실행 결과처럼 서로 다른 상태를 API와 데이터베이스로 연결하는 경험을 쌓았습니다.

기술 스택

React , JavaScript , Vite , Node.js , Express , PostgreSQL , Prisma , Firebase Authentication , Gemini API , Monaco Editor

서비스 데모 / MDB Academy

AI Pass

하나의 구독, 작업 목적에 따라 사용하는 공통 크레딧 | 개인 프로젝트

코딩, 영상, 음성 작업마다 AI 서비스를 따로 구독하는 부담에서 출발했습니다. 하나의 구독으로 받은 크레딧을 필요한 작업에 배분하는 통합 AI 서비스를 구상하고, 작업 공간, 견적, 승인, 정산 흐름을 프로토타입으로 구현했습니다.

실행 전 승인부터 정산까지

- 대화, 이미지, 영상 작업 선택, 모델 검색, 설정과 생성물 목록, 상세 화면을 개발했습니다.
- 프롬프트, 설정, 대화량을 반영한 견적과 만료, 실행 전 비용 확인 및 승인 검증 흐름을 구현했습니다.
- 중복 제출 방지 키와 작업 큐, 워커를 구성하고 작업 상태, 중단, 타임아웃을 처리했습니다.
- PostgreSQL 트랜잭션과 계정 잠금을 활용해 데모 크레딧의 예약, 정산, 환급을 관리했습니다.
- OpenAPI, Orval, Zod 기반 계약과 검증, 쿠키 기반 방문자별 데모 세션과 비공개 결과물 접근을 구성했습니다.

제품 비전과 현재 구현의 구분

Claude Code, Codex, Higgsfield, Runway, ElevenLabs는 향후 통합을 설명하는 예시이며 현재 모두 지원하는 것은 아닙니다. 결제, 구독 및 이미지, 영상 결과는 데모, 샘플입니다. 실제 호출 경로는 OpenRouter의 지정 무료 모델로 제한했으며, 배포 환경의 키 설정에 따라 실행 가능 여부가 달라집니다.

기술 스택

React , TypeScript , TanStack Query , Wouter , Express , PostgreSQL , Drizzle , OpenAPI , Orval , Zod , OpenRouter , Node 워커

프로토타입 데모 / AI Pass

경험을 다음 개발로 연결하기

기능 구현 , 데이터 흐름 , 검증 , 협업 , 배포

프로젝트를 거치며 화면과 서버를 각각 완성하는 것보다, 사용자의 요청이 어떤 데이터와 상태 변경으로 이어지는지 이해하는 일이 중요하다는 것을 배웠습니다. 생성 결과 편집과 저장, 실패 시 크레딧 반환, 인증과 편집 권한, 학습 활동 기록처럼 사용자 경험과 서버 상태가 만나는 부분을 직접 다뤘습니다.

개발 과정에서 쌓은 방식

- 사용자 입장에서 정상, 예외 흐름을 실행하고 실패 지점을 찾아 수정합니다.
- 복잡한 처리는 작은 함수와 명확한 책임으로 나눠 변경 흐름을 정리합니다.
- 익숙하지 않은 문제는 전문가의 조언과 팀 협업을 통해 구현 방법을 구체화합니다.
- AI 기능은 데이터, 지시문, API, 화면을 함께 연결하며 서비스 목적에 맞게 구성합니다.
- 배포 이후에도 검색 접근성, 인증 경계와 자동 검증을 점검합니다.

앞으로의 방향

전공 지식과 도메인 지식을 바탕으로 사용자 중심의 end - to - end 앱과 웹을 개발하고자 합니다. 요구사항을 이해하고 구현 가능한 구조로 바꾸는 능력을 실제 경험으로 다지며, 검증한 결과와 남은 한계를 분명하게 설명하는 엔지니어로 성장하겠습니다.

코드와 서비스 확인

Website / bakhtiyor.me

GitHub / BakhtiyorParpiboev

본 문서는 경력 연수, 이용자 수, 성능 개선율을 임의로 산정하지 않았습니다. 프로젝트별 진행 연월과 UITC 활동 연월은 확인 후 추가할 수 있습니다. 공개 링크는 구현 근거이며 모든 기능의 현재 실행 성공을 의미하지 않습니다.